TCS Coding Preparation – Complete Notes (Previous Year Questions)

These notes cover:

- Important **TCS coding patterns**
- **Array input techniques**
- **String problems**
- **Hashing / Map**
- **Greedy logic**
- **Matrix problems**
- **Sorting algorithms**
- **Dynamic array input without knowing size**

---

1. Valid String Problem (Star `*` and Hash `#`)

Problem Statement

You are given a string containing only:

```
*  (star)
#  (hash)
```

Task

Find the **minimum number of characters needed to make the string valid**.

Valid String Condition

A string is **valid if**

```
Number of *  =  Number of #
```

Output Rules

Condition	Output
Stars > Hash	Positive number
Hash > Stars	Negative number
Equal	0

Meaning of Output

- **Positive value Need more #**
- **Negative value Need more ***
- **Zero Already valid**

Example 1

```
Input:
***##

Stars = 3
Hash = 2
```

Difference

```
3 - 2 = 1
```

Output

```
1
```

Meaning Add **1 hash**

Example 2

```
Input:
*##
```

Stars = 1 Hash = 2

```
1 - 2 = -1
```

Output

```
-1
```

Meaning Need **1 star**

Example 3

```
Input:
**##
```

Stars = 2 Hash = 2

Output

```
0
```

Already valid.

Logic

Step-by-Step

1. Take input string.
2. Traverse string.
3. Count:
   - number of `*`
   - number of `#`
4. Return:

```
stars - hash
```

Algorithm

```
Input string S
countStar = 0
countHash = 0

for each character in string
   if '*'
      countStar++
   else
      countHash++

answer = countStar - countHash

print answer
```

Time Complexity

```
O(N)
```

Because we traverse the string once.

C++ Code

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {

    string s;
    getline(cin, s);

    int countStar = 0;
    int countHash = 0;

    for(int i = 0; i < s.length(); i++)
    {
        if(s[i] == '*')
            countStar++;
        else if(s[i] == '#')
            countHash++;
    }

    int result = countStar - countHash;

    cout << result;

    return 0;
}
```

Common Mistakes

Using `cin >> s` instead of `getline` Forgetting to handle both characters Incorrect subtraction order

---

Interview Questions

Q1 What is time complexity?

O(N)

Q2 Why use `getline()`?

Because string may contain spaces.

---

2. Count Elements Greater Than All Previous Elements

Problem

Given array:

```
arr = [7,4,8,2,9]
```

Count elements that are **greater than all previous elements**.

Example

```
7 4 8 2 9
```

Check step by step

Element	Check	Result
7	first element	count
4	4 > 7?	No
8	8 > 7	Yes
2	2 > 8?	No
9	9 > 8	Yes

Answer

```
3
```

Elements **7, 8, 9**

Brute Force Approach

Compare each element with all previous elements.

```
Time Complexity = O(N²)
```

Not good for large inputs.

Optimal Approach (Greedy)

Use a variable:

```
maximum
```

Track largest value seen so far.

Steps

1. First element always counted.

2. Store first element as `maximum`.
3. Traverse array.
4. If element > maximum

```
count++
maximum = element
```

Algorithm

```
count = 1
max = arr[0]

for i = 1 to n-1
   if arr[i] > max
      count++
      max = arr[i]

return count
```

Time Complexity

```
O(N)
```

C++ Code

```
#include <iostream>
using namespace std;

int countGreater(int arr[], int n)
{
```

```
    int count = 1;
    int maximum = arr[0];

    for(int i = 1; i < n; i++)
    {
        if(arr[i] > maximum)
        {
            count++;
            maximum = arr[i];
        }
    }

    return count;
}

int main()
{
    int n;
    cin >> n;

    int arr[n];

    for(int i = 0; i < n; i++)
        cin >> arr[i];

    cout << countGreater(arr, n);

    return 0;
}
```

Important Interview Concept

Greedy Approach

We only compare with the **largest previous element**, not every element.

---

3. Find Balloon Color Appearing Odd Times

Problem

You are given colors of balloons.

Example

```
R G B B G Y Y
```

Find the **color that appears odd number of times**.

Example

```
R G B B G Y Y
```

Count frequency

Color	Count
R	1
G	2
B	2
Y	2

Output

```
R
```

Method 1 — Hash Map (Frequency Count)

Use **Map / HashMap**

Map stores

```
Key    Value
Color    Frequency
```

Algorithm

```
create map

for each element
   map[element]++

for each element
   if frequency % 2 != 0
      return element
```

C++ Code

```cpp
#include <bits/stdc++.h>
using namespace std;

int main()
{
   int n;
   cin >> n;
```

```
    char arr[n];

    for(int i = 0; i < n; i++)
        cin >> arr[i];

    unordered_map<char,int> mp;

    for(int i = 0; i < n; i++)
        mp[arr[i]]++;

    for(int i = 0; i < n; i++)
    {
        if(mp[arr[i]] % 2 != 0)
        {
            cout << arr[i];
            return 0;
        }
    }

    cout << "All are even";
}
```

Method 2 — XOR Trick (Very Important)

If only **one element appears odd times**.

Use XOR property.

XOR Rules

```
A ^ A = 0
A ^ 0 = A
```

Example

```
R ^ R = 0
G ^ G = 0
B ^ B = 0
```

Remaining element odd one.

Algorithm

```
answer = 0

for each element
    answer = answer ^ element

print answer
```

Time Complexity

```
O(N)
```

Space Complexity

```
O(1)
```

Interview Questions

Q1 When use XOR method?

When **only one element occurs odd times**.

---

4. Monkey and Bananas Problem

Problem

Input

```
N    number of monkeys
K    bananas eaten by one monkey
J    peanuts eaten by one monkey
M    total bananas
P    total peanuts
```

Find number of monkeys left on tree.

Example

```
N = 10
K = 4
J = 3
M = 14
P = 9
```

Step 1

Monkeys eating bananas

```
14 / 4 = 3
```

Remaining bananas

```
14 % 4 = 2
```

Step 2

Monkeys eating peanuts

```
9 / 3 = 3
```

Remaining

```
9 % 3 = 0
```

Step 3

Total monkeys that ate

```
3 + 3 = 6
```

But bananas remain 2

Last monkey can eat remaining.

```
6 + 1 = 7
```

Step 4

Remaining monkeys

```
10 - 7 = 3
```

Output

```
3
```

C++ Code

```
#include <iostream>
using namespace std;

int main()
{
    int N,K,J,M,P;

    cin >> N >> K >> J >> M >> P;

    int bananaEat = M / K;
    int peanutEat = P / J;

    int remainingBanana = M % K;
    int remainingPeanut = P % J;

    int eaten = bananaEat + peanutEat;

    if(remainingBanana != 0 || remainingPeanut != 0)
        eaten++;

    int result = N - eaten;

    cout << result;

    return 0;
}
```

---

5. Sort Array of 0 1 2

Problem

Sort array containing only

```
0 1 2
```

Example

```
1 0 0 1 2 0 1
```

Output

```
0 0 0 1 1 1 2
```

Method 1 — Counting Method

Count:

```
count0
count1
count2
```

Then rewrite array.

Time Complexity

```
O(N)
```

Method 2 — Dutch National Flag Algorithm (IMPORTANT)

Invented by **Edsger Dijkstra**

Three pointers

```
low
mid
high
```

Rules

```
0    swap with low
1    move mid
2    swap with high
```

Diagram

```
[0 ... low-1] = 0
[low ... mid-1] = 1
[mid ... high] = unknown
[high+1 ... end] = 2
```

Algorithm

```
low = 0
mid = 0
high = n-1

while(mid <= high)

    if arr[mid] == 0
        swap(arr[mid], arr[low])
        low++
        mid++
```

```
else if arr[mid] == 1
    mid++

else
    swap(arr[mid], arr[high])
    high--
```

Time Complexity

```
O(N)
```

Space

```
O(1)
```

C++ Code

```
#include <iostream>
using namespace std;

void sort012(int arr[], int n)
{
    int low = 0;
    int mid = 0;
    int high = n - 1;

    while(mid <= high)
    {
        if(arr[mid] == 0)
        {
            swap(arr[mid], arr[low]);
```

```
            low++;
            mid++;
        }
        else if(arr[mid] == 1)
        {
            mid++;
        }
        else
        {
            swap(arr[mid], arr[high]);
            high--;
        }
    }
}
```

---

6. Longest Common Prefix

Problem

```
["flower","flow","flight"]
```

Output

```
fl
```

Idea

1. Sort strings
2. Compare **first and last string**

Why?

Because after sorting

```
common prefix between first and last
= common prefix of all
```

Algorithm

```
sort array

first = arr[0]
last = arr[n-1]

compare characters

until mismatch
```

C++ Code

```
string longestCommonPrefix(vector<string>& strs)
{
    sort(strs.begin(), strs.end());

    string first = strs[0];
    string last = strs[strs.size()-1];

    string ans = "";

    for(int i=0;i<first.length();i++)
    {
        if(first[i] == last[i])
            ans += first[i];
        else
            break;
```

```
    }

    return ans;
}
```

Time Complexity

```
O(N log N)
```

---

7. Count Sundays in a Month

Input

```
Starting day
Total days
```

Example

```
Start = Monday
Days = 13
```

Days until Sunday

Day	Days to Sunday
Monday	6
Tuesday	5
Wednesday	4
Thursday	3
Friday	2
Saturday	1
Sunday	0

Formula

```
answer = 1 + (remainingDays / 7)
```

---

8. Matrix Problem – Row with Maximum 1s

Given matrix

```
0 1 0
1 1 0
1 1 1
```

Row with maximum ones

```
Row 3
```

Logic

1. Traverse matrix
2. Count ones in each row
3. Track maximum

Code

```
int maxRow = -1;
int maxCount = 0;
```

```
for(int i=0;i<rows;i++)
{
    int count = 0;

    for(int j=0;j<cols;j++)
    {
        if(matrix[i][j] == 1)
            count++;
    }

    if(count > maxCount)
    {
        maxCount = count;
        maxRow = i;
    }
}

cout << maxRow + 1;
```

---

9. Taking Array Input Without Knowing Size

Important for **TCS exam**

User input

```
2,3,4,5,6
```

or

```
2 3 4 5 6
```

Method (C++)

Steps

1. Take full input as **string**
2. Convert to **stringstream**
3. Split by delimiter
4. Convert to integer
5. Push in vector

Code

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    string s;
    getline(cin, s);

    stringstream ss(s);

    vector<int> arr;

    string token;

    while(getline(ss, token, ','))
    {
        int num = stoi(token);
        arr.push_back(num);
    }

    for(int x : arr)
        cout << x << " ";
}
```

Important Interview Topics From Lecture

You should know:

Array

- static input
- dynamic input

String

- prefix
- traversal

Hashing

- map
- unordered_map

Greedy

- maximum tracking

Bitwise

- XOR trick

Sorting

- Dutch National Flag

Matrix

- row traversal
- nested loops

Common Mistakes in TCS Exam

Using inbuilt sort when restricted Not handling edge cases Incorrect indexing Using $O(N^2)$ when O(N) possible Forgetting input format

---

Important Interview Questions

1 What is hashing?

Technique to store **key-value pairs**.

---

2 What is greedy approach?

Choose **best option at current step**.

---

3 XOR properties?

```
A ^ A = 0
A ^ 0 = A
```

---

4 Time complexity of Dutch Flag?

```
O(N)
```

---

5 Difference map vs unordered_map?

Map	Unordered Map
Sorted	Not sorted
O(log n)	O(1)

If you want, I can also create **TCS NQT SUPER NOTES (Top 30 Coding Questions + Patterns)** which will help you **clear TCS coding round easily.**